

Solar Fleet Operations & Smart AI Dispatcher

สร้างเว็บมอนิเตอร์โซลาร์ฟาร์มแบบเรียลไทม์ พร้อมระบบ AI ประเมินจุดคุ้มทุนสิ่งล้างแผง (Cost of Inaction ROI) — ดึงข้อมูลจาก Solar IoT API

🕒 เวลาโดยประมาณ: 90–120 นาที | 🎯 เป้าหมาย: ผ่านครบ 6 requirement ขั้นต่ำ | 💻 รูปแบบ: Vibe Coding

เข้าใจภาพใน 30 วินาที

นี่คือสิ่งที่เกิดขึ้นตั้งแต่ Solar IoT API ส่งข้อมูลเซนเซอร์มา จนถึงระบบ AI ประเมินความสูญเสียสะสม และแนะนำให้ผู้จัดการกองส่งทีมล้างแผงทันที — ทุกอย่างเกิดขึ้นแบบ Data-Driven และอัตโนมัติ

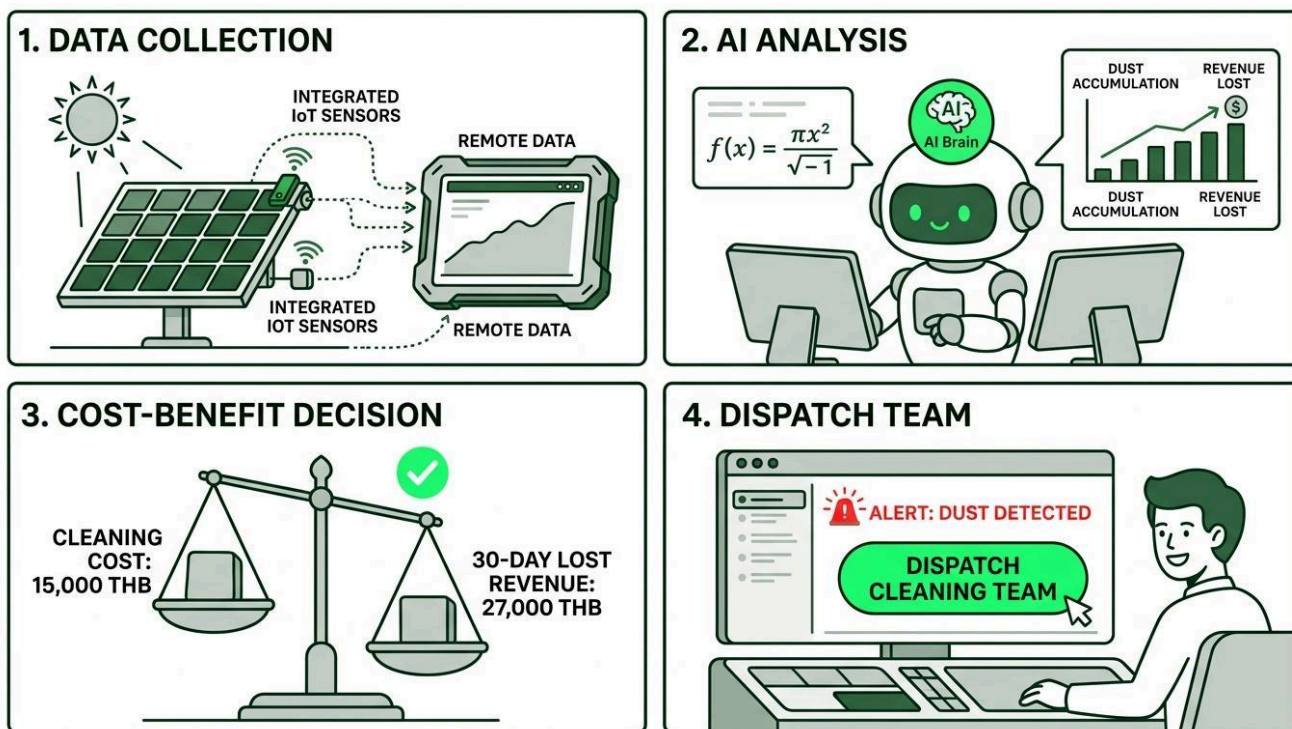


Figure 1: ตัวอย่างจริง แสดงขั้นตอนการดึง Telemetry, ตรวจจับฝุ่น & รายได้สูญเสีย, AI ชั่งน้ำหนักจุดคุ้มทุน และหน้าแจ้งเตือน

1 ดึงข้อมูลเซนเซอร์	2 ตรวจจับฝุ่น & สูญเสีย	3 AI วิเคราะห์จุดคุ้มทุน	4 แจ้งเตือน
เรียก Telemetry API รับค่า W/m², กำลังผลิต kW, อุณหภูมิ และพลังงานสะสมวันนี้	คำนวณ Soiling Loss % เทียบแดดจริง และตีมูลค่าเป็น 'เงินบาทที่หายไปต่อวัน'	ประเมิน 30-Day Cost of Inaction เทียบกับค่าล้างแผง 15,000 บาท ว่าคุ้มค่าล้างหรือไม่	แสดงผลบนหน้าจอ Dashboard สีเขียว/เหลือง/แดง แสดงว่าโซลาร์ไหนต้องส่งคนไปล้าง

1. โจทย์นี้ต้องการอะไร

บริษัทของเราบริหารโซลาร์เซลล์ 10 โซลาร์ทั่วประเทศ (โรงงาน, คลังห้องเย็น, โรงพยาบาล, อาคารสำนักงาน) แต่ละแห่งมีผู้ดูแลและพฤติกรรมใช้ไฟต่างกัน แต่การส่งทีมไปล้างแผงมีค่าใช้จ่ายสูงถึง 15,000 บาท

ทำไมเรื่องนี้ถึงต้องใช้ระบบ ช่วยตัดสินใจ?

- ล้างเร็วไป = ขาดทุน: โซลาร์น้อยๆ ปล่อยทิ้งไว้ 30 วันเสียแค่ 2,400 บาท แต่รับจ่ายค่าล้าง 15,000 บาท
- ล้างช้าไป = เสียหายหนัก: โซลาร์สกปรกสะสมสกปรก ปล่อยทิ้งไว้ 30 วันเสียเงิน 36,000 บาท (ล้างวันนี้ประหยัดได้ 21,000 บาท)

สิ่งที่ต้องสร้าง: Operations Dashboard ที่รวมข้อมูล 10 โซลาร์ไว้ทีเดียว มี AI คำนวณความคุ้มค่าแบบอัตโนมัติ พร้อมปุ่มสั่งส่งทีมล้างแผง และปุ่มจำลองพายุฝุ่นเพื่อทดสอบระบบ

สิ่งที่นักเรียนต้องสร้าง: คือเว็บแอปพลิเคชัน Operations Dashboard อัจฉริยะที่เชื่อมต่อกับ Solar IoT API เพื่อเฝ้าระวังโซลาร์ฟาร์ม 10 โซลาร์แบบ Real-time และมีระบบ AI ที่ช่วยวิเคราะห์ว่า: 'โซลาร์ไหนควรส่งล้างทันที และโซลาร์ไหนยังไม่จำเป็นต้องล้าง เพื่อให้เกิดผลประโยชน์สูงสุด?'

2. สิ่งที่คุณได้รับ (API Specification & Data Fields)

เพื่อจำลองสภาพแวดล้อมเสมือนจริงของระบบ Industrial IoT โดยสามารถทดสอบสถานการณ์จำลอง (เช่น พายุฝุ่น หรือการล้างแผง) ได้อย่างปลอดภัยและเห็นผลทันที เราจึงได้สร้างแบบจำลองระบบโซลาร์ฟาร์มจำนวน 10 โครงการ โครงการละ 1,000 แผง กำลังแผงละ 550-740 วัตต์ (คิดเป็นกำลังการผลิตติดตั้ง 550-740 kWp ต่อโครงการ)

ผู้สอนได้ Deploy ระบบ Solar IoT Mock API ขึ้นสู่ Cloud เรียบร้อยแล้ว:

Base URL: <https://solar-test-api.onrender.com>

Authentication: ส่ง Header x-api-key: <your_group_key> ในทุก Request

2.1 สรุปภาพรวม Endpoints ทั้งหมด

Method & Endpoint	ประเภท	รายละเอียดการใช้งาน	ผลลัพธ์หลัก
GET /api/v1/{group_id}/sites	Fleet Info	ดึงรายชื่อและข้อมูลพื้นฐานของทั้ง 10 โซลาร์	10 sites metadata
GET /api/v1/{group_id}/telemetry	Real-time	ดึงข้อมูลเซนเซอร์สดรวมทุกโซลาร์พร้อมกัน (Snapshot ล่าสุด)	Sensor ทั้ง 10 โซลาร์
GET /api/v1/{group_id}/telemetry/{site_id}	Real-time	ดึงเซนเซอร์สดเจาะจงเฉพาะโซลาร์ที่ต้องการ	Sensor โซลาร์เดียว
GET /api/v1/{group_id}/history/last/{days}	History	ดึงประวัติย้อนหลังรายวัน (1 ถึง 30 วัน)	ประวัติรายวัน 10 โซลาร์
GET /api/v1/{group_id}/history/monthly/{months}	History	ดึงประวัติย้อนหลังรายเดือน (1 ถึง 12 เดือน)	ประวัติรายเดือน 10 โซลาร์

2.2 รายละเอียดข้อมูลในเส้นหลัก: GET /api/v1/{group_id}/telemetry

เส้นนี้คืนค่า Array ของเซนเซอร์สด ณ ปัจจุบันของทุกโซลาร์ ประกอบด้วยข้อมูลดิบทางวิศวกรรมที่ต้องนำมาประมวลผล:

ชื่อฟิลด์ (JSON Key)	ชนิดข้อมูล	หน่วย	ความหมายและการนำไปใช้งาน
site_id	string	-	รหัสประจำโซลาร์ เช่น 'SOLAR-BKK-01' ถึง 'SOLAR-PTM-10'

ชื่อฟิลด์ (JSON Key)	ชนิดข้อมูล	หน่วย	ความหมายและการนำไปใช้งาน
site_name	string	-	ชื่อไซต์ เช่น 'Bangkok Urban Factory Rooftop'
timestamp	ISO8601	UTC	เวลาที่บันทึกข้อมูลเซนเซอร์ล่าสุด
irradiance_w_m2	float	W/m²	ความเข้มแสงแดดสดขณะนั้น (ใช้คำนวณ Expected Power) *กลางคืน = 0*
sun_hours_h	float	ชม. (h)	ชั่วโมงแดดสะสมจริงตั้งแต่เช้าจนถึงปัจจุบัน
panel_temp_c	float	°C	อุณหภูมิผิวแผงโซลาร์เซลล์ (ปกติ 40°C - 65°C)
pv_power_kw	float	kW	กำลังไฟฟ้าที่แผงผลิตได้จริง ณ ขณะนั้น (Actual Power)
load_power_kw	float	kW	กำลังไฟฟ้าที่โรงงาน/อาคารกำลังเปิดใช้งานอยู่ ณ ขณะนั้น
actual_yield_kwh	float	kWh	พลังงานไฟฟ้าสะสมที่ผลิตได้จริงในวันนี้ (ใช้คำนวณเงินรายได้สะสม)
consumed_kwh	float	kWh	พลังงานไฟฟ้าที่โรงงานใช้ทั้งหมดสะสมในวันนี้
exported_kwh	float	kWh	ไฟฟ้าโซลาร์ส่วนเกินที่ส่งออกขายคืนการไฟฟ้าสะสมวันนี้ (Tariff 2.20 บ.)
imported_kwh	float	kWh	ไฟฟ้าที่โรงงานต้องซื้อเพิ่มจากการไฟฟ้าสะสมวันนี้

ตัวอย่าง Response จริงจาก GET .../telemetry (1 ไซต์):

```
{
  "site_id": "SOLAR-BKK-01",
  "site_name": "Bangkok Urban Factory Rooftop",
  "timestamp": "2026-09-12T07:45:00.000Z",
  "irradiance_w_m2": 845.2,
  "sun_hours_h": 4.12,
  "panel_temp_c": 51.8,
  "pv_power_kw": 412.50,
  "load_power_kw": 580.20,
  "actual_yield_kwh": 1840.50,
  "consumed_kwh": 2450.00,
  "exported_kwh": 0.00,
  "imported_kwh": 609.50
}
```

2.3 รายละเอียดข้อมูลประวัติย้อนหลัง: GET .../history/last/{จำนวน} และ .../history/monthly/{จำนวน}

เส้นประวัติย้อนหลังรายวัน (1-30 วัน) และรายเดือน (1-12 เดือน) เป็นข้อมูลที่สมบูรณ์แล้วของแต่ละช่วงเวลา (Completed Records) เหมาะสำหรับนำมาพล็อตกราฟแนวโน้ม และใช้เป็น Baseline คำนวณช่วงกลางคืน โดยมีรายละเอียดฟิลด์ดังนี้:

ชื่อฟิลด์ (JSON Key)	ชนิดข้อมูล	หน่วย	ความหมายและการนำไปใช้งาน
date / month	string	YYYY-MM-DD / YYYY-MM	วันที่ (เส้นรายวัน /last ใช้ key 'date') หรือเดือน (เส้นรายเดือน /monthly ใช้ key 'month')
site_id / site_name	string	-	รหัสและชื่อไซต์โซลาร์ เช่น 'SOLAR-BKK-01'
region	string	-	ภูมิภาค เช่น 'Bangkok (Urban)', 'Rayong (Industrial Estate)'
weather_condition	string	-	สภาพอากาศในวันนั้น ('hot', 'rain', 'cool', 'rainy') *(มีเฉพาะในเส้นรายวัน /last เท่านั้น)*
days_recorded	integer	วัน	จำนวนวันที่บันทึกในเดือนนั้น เช่น 30 หรือ 31 วัน *(มีเฉพาะในเส้นรายเดือน /monthly เท่านั้น)*
avg_irradiance_w_m2	float	W/m²	ความเข้มแสงแดดเฉลี่ยของวันนั้น หรือค่าเฉลี่ยตลอดทั้งเดือน
avg_panel_temp_c	float	°C	อุณหภูมิแผงเฉลี่ยของวันนั้น หรือค่าเฉลี่ยตลอดทั้งเดือน
sun_hours_h	float	ชม.	จำนวนชั่วโมงแดดเต็มวัน (รายวัน) หรือผลรวมชั่วโมงแดดสะสมทั้งเดือน (รายเดือน SUM)
expected_yield_kwh	float	kWh	หน่วยไฟฟ้าที่ควรผลิตได้ตามแคต (รายวันเป็นยอดวันนั้น / รายเดือนเป็นผลรวมทั้งเดือน SUM)
actual_yield_kwh	float	kWh	หน่วยไฟฟ้าที่ผลิตได้จริง (รายวันเป็นยอดวันนั้น / รายเดือนเป็นผลรวมทั้งเดือน SUM) -> ส่วนต่างคือฝุ่น
consumed_kwh	float	kWh	การใช้ไฟฟ้าทั้งหมดของสถานที่ (ยอดวันนั้น หรือผลรวมทั้งเดือน)
exported_kwh	float	kWh	ไฟฟ้าส่วนเกินที่ขายคืนเข้า Grid (ยอดวันนั้น หรือผลรวมทั้งเดือน)
imported_kwh	float	kWh	ไฟฟ้าที่ซื้อเพิ่มจากการไฟฟ้า (ยอดวันนั้น หรือผลรวมทั้งเดือน)

ตัวอย่าง Response จริง (1) เส้นรายวัน GET .../history/last/1/{site_id}:

```
{
  "date": "2026-09-11",                // <-- ใช้ "date" ในเส้นรายวัน
  "site_id": "SOLAR-BKK-01",
  "site_name": "Bangkok Urban Factory Rooftop",
  "region": "Bangkok (Urban)",
  "weather_condition": "hot",          // <-- มีเฉพาะเส้นรายวัน
  "avg_irradiance_w_m2": 780.5,
  "avg_panel_temp_c": 49.2,
  "sun_hours_h": 5.10,
  "expected_yield_kwh": 3003.90,
  "actual_yield_kwh": 2553.30,
  "consumed_kwh": 18200.00,
  "exported_kwh": 0.00,
  "imported_kwh": 15646.70
}
```

ตัวอย่าง Response จริง (2) เส้นรายเดือน GET .../history/monthly/1/{site_id}:

```
{
  "month": "2026-08",                 // <-- ใช้ "month" (ไม่มี date)
  "site_id": "SOLAR-BKK-01",
  "site_name": "Bangkok Urban Factory Rooftop",
  "region": "Bangkok (Urban)",
  "days_recorded": 31,                // <-- มีเฉพาะเส้นรายเดือน
  "avg_irradiance_w_m2": 670.2,
  "avg_panel_temp_c": 47.8,
  "sun_hours_h": 142.50,              // ผลรวมแดดทั้งเดือน (SUM)
  "expected_yield_kwh": 83920.00,     // ผลรวมหน่วยไฟทั้งเดือน (SUM)
  "actual_yield_kwh": 71332.00,
  "consumed_kwh": 564200.00,
  "exported_kwh": 0.00,
  "imported_kwh": 492868.00
}
```

3. ข้อกำหนดขั้นต่ำ (Core Requirements)

- ☐ Requirement 1: ระบบสามารถแจ้งเตือนผู้ใช้งานได้ ว่าไซต์ไหนต้องส่งคนเข้าไปล้าง

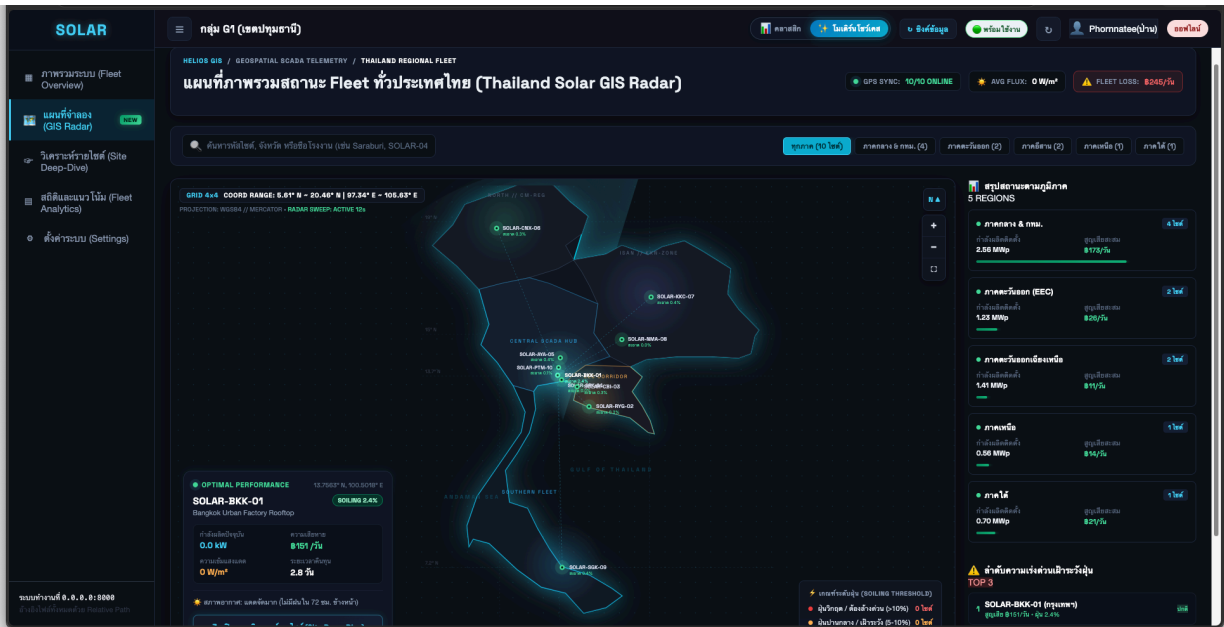
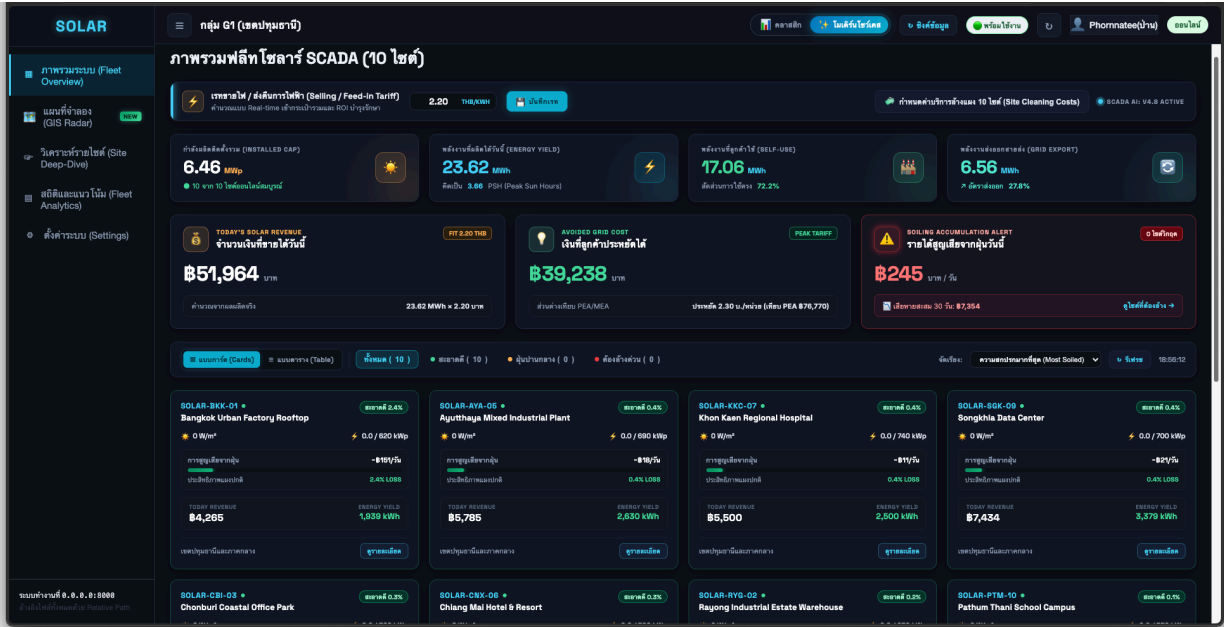
ประเมินจากกำลังไฟที่ควรผลิตได้จากความเข้มแสงในขณะนั้น หากกำลังไฟผลิตได้น้อยกว่าที่ควรยาวนาน ก็อาจสันนิษฐานได้ว่าไซต์นั้น มีแผงโซลาร์เซลล์ที่ต้องการการทำความสะอาด

Requirement 2: ข้อมูล Dashboard มีความเข้าใจและอ่านง่าย

ตกแต่งตามความชอบ ไม่จำกัดรูปแบบ ขอแค่เป็นระบบที่ผู้สามารถใช้งานได้ง่าย

ตัวอย่าง

https://solar.ufochannelthai.workers.dev/



4. สูตรคำนวณและเกณฑ์ทางธุรกิจ (Business Logic)

ชื่อสูตร / ตัวชี้วัด	สมการคำนวณ (Formula)	คำอธิบายทางธุรกิจ
1. Expected Power (kW)	$\text{Capacity (kWp)} \times (\text{Irradiance} / 1000) \times 0.95$	กำลังไฟฟ้าที่แผงควรผลิตได้ตามทฤษฎีเมื่อแดดส่อง (หัก System Derate 5%)

ชื่อสูตร / ตัวชี้วัด	สมการคำนวณ (Formula)	คำอธิบายทางธุรกิจ
2. Soiling Loss (%)	$\frac{(\text{Expected} - \text{Actual})}{\text{Expected}} \times 100$	สัดส่วนประสิทธิภาพที่สูญเสียไปเนื่องจากฝุ่นบดบังแสงแดด
3. Daily Lost (kWh)	$\text{Capacity (kWp)} \times 4.5\text{h} \times 0.95 \times \frac{(\text{Loss\%} / 100)}$	ประเมินหน่วยไฟฟ้าที่หายไปในวันเต็ม (อ้างอิงแดดเฉลี่ยไทย 4.5 ชม./วัน)
4. Daily Lost (THB)	$\text{Daily Lost (kWh)} \times \text{Tariff (บาท/kWh)}$	แปลงหน่วยไฟฟ้าที่หายไปเป็นเงินบาท (ใช้ Tariff 2.20 หรือคำนวณตามสัดส่วน 4.50 บ.)
5. 30-Day Inaction Loss	$\text{Daily Lost (THB)} \times 30 \text{ วัน}$	คาดการณ์ความเสียหายสะสม 30 วันข้างหน้า หากผู้จัดการตัดสินใจ 'ไม่ทำอะไรเลย'
6. Dispatch Decision	If (30-Day Loss > 15,000 THB) -> สั่ง ล้างพื้นที่ $\text{Net Saving} = 30\text{-Day Loss} - 15,000 \text{ THB}$	หลักคิดเศรษฐศาสตร์: ถ้าปล่อยไว้ 30 วันแล้วเสียเงินมากกว่าค่าล้าง 15,000 บ. ให้สั่งทีมล้างพื้นที่
7. Night Fallback Rule	If (Irradiance < 150 W/m²): ใช้ Loss% และ Lost THB ของเมื่อวาน	กฎความปลอดภัย: ป้องกันระบบแจ้งเตือนผิดพลาดช่วงกลางคืนที่ไม่มีแสงแดด

5. แนวทางการ Prompt สำหรับ Vibe Coding (AI Guide)

💡 จังหวะที่ 1: แจ้งความต้องการของระบบ

เป้าหมาย: สร้างเว็บแอปพลิเคชันที่แสดงข้อมูลของทั้งโซลาร์ฟาร์มที่ติดตั้งแผงโซลาร์เซลล์

สร้างโปรเจกต์ชื่อ "solar-fleet-dashboard" เป็น folder ใหม่ใน folder ปัจจุบัน โดยทำตาม @AGENTS.md

ฉันต้องการให้ app นี้:

- จัดการการตั้งค่าระบบผ่านไฟล์ .env ได้ (ตรวจสอบด้วยว่าค่าที่จำเป็นถูกตั้งไว้ครบและไม่ใช่ว่าง)
- ดึงข้อมูลโซลาร์เซลล์มาจาก API ที่ฉันมีให้ มาแสดงภาพรวมของแต่ละโซลาร์ (มี 10 โซลาร์)
- ระบบมีการคำนวณบอกรายได้จากการส่งไฟที่ผลิตเกิน ไปขายคืนให้การไฟฟ้า
- ระบบมีการคำนวณและประเมินว่าโซลาร์ไหนต้องเข้าไปทำความสะอาดเนื่องจากกำลังการผลิตไฟโดนฝุ่นขัดขวาง

หากมีตรงไหนสงสัยหรือยังไม่ครบถ้วน ช่วยถามฉันกลับมาด้วย เพื่อความสมบูรณ์ของระบบ

👉 เป้าหมาย: ให้ AI ทราบว่าเราต้องการระบบแบบไหน

💡 จังหวะที่ 2: วิธีการเชื่อมต่อ API

ข้อมูลการเชื่อมต่อ:

- Base URL: "https://solar-test-api.onrender.com"
- Group ID: "g..." (เปลี่ยนตามกลุ่มที่ได้รับ)
- Headers: {"x-api-key": "xxxxxxxxx"}

Endpoints ที่ต้องใช้:

- ข้อมูลโซลาร์: GET /api/v1/{group_id}/sites
- ข้อมูลเซนเซอร์สด: GET /api/v1/{group_id}/telemetry
- สั่งทีมล้างแผง: POST /api/v1/{group_id}/telemetry/{site_id}/clean
- จำลองพายุฝุ่น: POST /api/v1/simulate/dust-storm/{group_id}

เป้าหมาย: ให้ AI สร้าง API Client และดึงข้อมูลมาแสดงบนหน้าจอให้สำเร็จก่อน

👉 เป้าหมาย: ให้ AI รู้วิธีเชื่อมต่อ API เพื่อดึงข้อมูลมาใช้งานได้